SQL Forensics Investigating Security Anomalies and Access Logs

Project description

In this project, I used SQL queries to investigate potential security incidents by analyzing access logs and employee data. My goal was to filter and retrieve specific information from the `log_in_attempts and employees` tables to identify vulnerabilities and segment data for system updates. Through this activity, I demonstrate my ability to apply complex filtering logic and ensure the integrity of organizational information.

Recover after hours of failed login attempts

To investigate a possible incident that occurred outside of working hours, I leaked the access logs to

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_time > '18:00' AND success = 0;
+----------+----------+------------+------------+---------+---------
--------+---------+
| event_id | username | login_date | login_time | country | ip_addre
ss       | success |
```

identify failed attempts made after 18:00.

1. **Consulta:** `SELECT * FROM log_in_attempts WHERE login_time > '18:00' AND success = 0;`
2. **Explanation:** This query filters **date and time** data using the greater than operator (>) to identify records after '18:00'. In addition, it uses the **logical operator AND** to ensure that two conditions are met simultaneously: that the access is late and that the `success` field is `0` (indicating a failure).

Recover login attempts on specific dates

Investigate a suspicious event that occurred between May 8 and May 9, 2022 by analyzing access dates.

1. **Query:** `SELECT * FROM log_in_attempts WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';`

```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';
```

- **Explicación:** Utilicé el operador **OR** para recuperar registros que coincidieran con cualquiera of the two specific dates. Date filtering requires the use of single quotation marks for the SQL engine to properly process the date string format.

Recover login attempts outside of Mexico

To reduce the noise in the research, I filtered out login attempts that didn't originate in Mexico.

- **Query:** `SELECT * FROM log_in_attempts WHERE NOT country LIKE 'MEX%';`
-
```
MariaDB [organization]> SELECT *
    ->
    -> FROM log_in_attempts
    ->
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+---------
--------+---------+
| event_id | username | login_date | login_time | country | ip_addre
ss       | success |
```
-

1. **Explanation:** I used the **NOT** operator to negate the condition and exclude specific records. To identify variations like 'MEX' or 'MEXICO', I used the **LIKE** keyword along with the percentage (%) wildcard, which allows you to search for a **pattern** that starts with those characters regardless of the final length of the string.

Recover employees in Marketing

I identified Marketing employees in the East Building to perform targeted security updates.

- **Query:** `SELECT * FROM employees WHERE department = 'Marketing' AND office LIKE 'East%';`
```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%'
    -> ;
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
```
-

1. **Explanation:** This query combines the **AND** operator to validate the exact department and the **LIKE operator** with the wildcard % to filter offices in the East building (patterns starting with 'East').

Recover employees in Finance or Sales

Segmented the information for updates in the Sales and Finance departments.

- **Query:** `SELECT * FROM employees WHERE department = 'Finance' OR department = 'Sales';`

```
MariaDB [organization]> SELECT *
    ->
    -> FROM employees
    -> WHERE department = 'Finance' OR department = 'Sales';
+-------------+-------------+----------+------------+-------------+
| employee_id | device_id   | username | department | office      |
```

1. **Explanation:** I used the **OR** operator to capture employees from both departments. It was necessary to specify the `department` column for both conditions for the join logic to work correctly.

Recover all non-IT employees.

Finally, I identified non-technical team employees to complete pending updates.

1. **Query:** `SELECT * FROM employees WHERE NOT department = 'Information Technology';`

```
MariaDB [organization]> SELECT * FROM employees
    ->
    -> WHERE NOT department = 'Information Technology';
+-------------+-------------+----------+----------------+---------
----+
| employee_id | device_id   | username | department     | office
    |
```

2. **Explanation:** The **NOT** operator was used to exclude employees from the IT department, allowing resources to be focused on the rest of the organization.

Summary

In this project, I applied a series of advanced SQL filters to investigate security incidents and manage system updates. By strategically using operators such as **AND, OR, NOT** and patterns with **LIKE**, I was able to extract accurate information from large data sets. These skills are critical for a security analyst looking to protect information assets through detailed log and asset analysis.